13

抽象：地址空间

早年间，构建计算机系统很容易。你问为什么？因为用户期望不高。正是那些该死的用户，带着他们对“易用”“高性能”“可靠”等等的期望，才真正惹出了这么多头疼的事。下次你遇到这样的计算机用户，记得为他们惹出的所有麻烦向他们道声谢。

13.1 早期系统

从内存的角度看，早期的机器并没有为用户提供多少抽象。基本上，机器的物理内存看起来就像图 13.1（第 1 页）那样。

操作系统是一组例程（其实就是一个库），放在内存里（本例中从物理地址 0 开始）；内存里还有一个正在运行的程序（一个进程），它当时就放在物理内存中（本例中从物理地址 64k 开始），并使用其余所有内存。这里几乎没有什么假象，用户对操作系统也没什么期望。那时候操作系统开发者的日子可真轻松，不是吗？

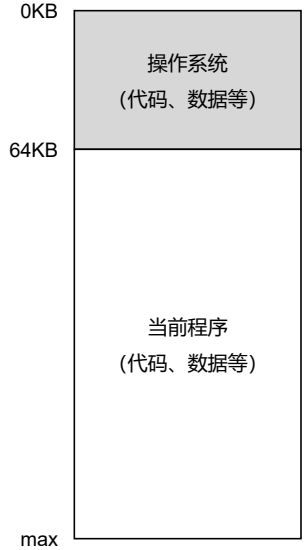


图 13.1：**操作系统：早期**

13.2 多道程序设计和分时

过了一段时间，由于机器很贵，人们开始更有效地共享机器。于是**多道程序设计**（multiprogramming）时代诞生了 [DV66]：在某一时刻，有多个进程准备运行，操作系统在它们之间切换，比如当一个进程决定执行 I/O 时就切换。这样做提高了 CPU 的有效**利用率**（utilization）。在那个年代，每台机器都要花费几十万甚至上百万美元（你还觉得你的 Mac 贵！），所以这种**效率**（efficiency）上的提升尤为重要。

然而，没过多久，人们开始对机器提出更多要求，**分时**（time sharing）时代随之诞生 [S59, L60, M62, M83]。具体来说，许多人意识到了批处理计算的局限，尤其是对程序员自己而言 [CV65]：他们受够了漫长（因而低效）的“编程—调试”循环。**交互性**（interactivity）变得重要起来，因为可能有许多用户同时使用一台机器，每个人都在等待（或者说盼望）自己当前正在执行的任务能及时给出响应。

实现分时的一种方法是：让一个进程运行一小会儿，让它完全访问所有内存（图 13.1），然后停下它，把它的全部状态保存到某种磁盘上（包括所有物理内存），再加载另一个进程的状态，让它运行一会儿，这样就实现了某种粗糙的机器共享 [M+63]。

遗憾的是，这种方法有一个大问题：它实在太慢了，尤其是随着内存越来越大。保存和恢复寄存器级的状态（PC、通用寄存器等）相对较快，但把内存的全部内容保存到磁盘上，性能就差得惨不忍睹。因此，我们更愿意做的是，在进程之间切换时把它们留在内存里，让操作系统能高效地实现分时（如图 13.2 所示，见第 3 页）。

在这张图中，有三个进程（A、B 和 C），每个进程都从 512 KB 的物理内存中分到一小块。假设只有一个 CPU，操作系统选择运行其中一个进程（比如 A），其他进程（B 和 C）则在就绪队列里等待运行。

随着分时越来越流行，你大概能猜到，人们对操作系统提出了新的要求。尤其是，允许多个程序同时驻留在内存中，使得**保护**（protection）成了一个重要问题：你可不希望一个进程能读取，或者更糟，能写入其他进程的内存。

13.3 地址空间

然而，我们必须把那些烦人的用户放在心上，而要做到这一点，操作系统就得为物理内存创建一个**易于使用**（easy to use）的抽象。我们把这个抽象叫作**地址空间**（address space），它是运行中的程序眼中系统内存的样子。理解操作系统对内存的这一基本抽象，是理解内存如何被虚拟化的关键。

进程的地址空间包含运行中的程序的全部内存状态。例如，程序的**代码**（code，即指令）必须放在内存中的某个地方，因此它们在地址空间里。程序运行时，用**栈**（stack）来记录自己在函数调用链中所处的位置，分配局部变量，以及向例程传递参数和返回值。最后，**堆**（heap）用于动态分配、由用户管理的内存，比如在 C 中调用 `malloc()`，或者在 C++、Java 这类面向对象语言中调用 `new` 所得到的内存。当然，地址空间里还有其他东西（例如静态初始化的变量），不过现在我们只假设有这三个部分：代码、栈和堆。

 © 2008–23, ARPACI-DUSSEAU



图 13.2：**三个进程共享内存**

在图 13.3（第 4 页）的例子中，我们有一个很小的地址空间（只有 16 KB）[1]。程序代码位于地址空间的顶部（本例中从 0 开始，挤在地址空间的第一个 1K 里）。代码是静态的（因此很容易放进内存），所以我们可以把它放在地址空间的顶部，并且知道程序运行时它不会再需要更多空间。

接下来是地址空间中在程序运行时可能增长（和收缩）的两个区域：堆（在顶部）和栈（在底部）。之所以这样放，是因为二者都希望能够增长；把它们放在地址空间的两端，就能允许这种增长：它们只需要朝相反的方向增长。因此，堆紧接在代码之后开始（在 1 KB 处），向下增长（比如用户通过 `malloc()` 申请更多内存时）；栈从 16 KB 处开始，向上增长（比如用户进行过程调用时）。不过，栈和堆的这种放法只是一种约定；只要你愿意，也可以用别的方式安排地址空间（我们后面会看到，当多个**线程**（threads）共存于一个地址空间中时，就再也没有像这样划分地址空间的好办法了，唉）。

当然，我们描述地址空间时，描述的是操作系统提供给运行中程序的**抽象**（abstraction）。程序其实并不在物理地址 0 到 16 KB 的内存里；它被加载到某个（或某些）任意的物理地址上。看看图 13.2 中的进程 A、B 和 C，你就能看出每个进程是怎样被加载到内存中不同地址上的。于是问题来了：

[1]我们会经常使用这样的小例子，因为（a）画一个 32 位的地址空间很麻烦，而且（b）算起来更难。我们喜欢简单的算术。

 © 2008–23, ARPACI-DUSSEAU

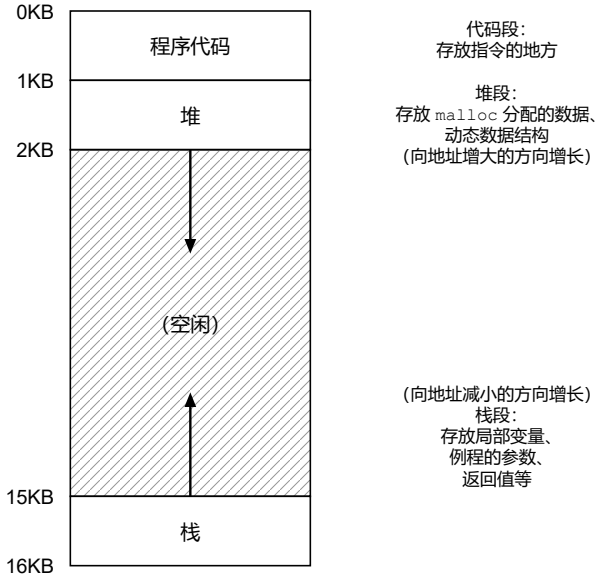


图 13.3：**一个地址空间示例**

关键问题：如何虚拟化内存
操作系统怎样在单一的物理内存之上，为多个运行中的进程（它们共享这块内存）构建出这样一个私有的、可能很大的地址空间抽象？

操作系统这样做时，我们就说操作系统在**虚拟化内存**（virtualizing memory），因为运行中的程序以为自己被加载到了某个特定的地址（比如 0），并拥有一个可能非常大的地址空间（比如 32 位或 64 位）；而实际情况却大不相同。

例如，当图 13.2 中的进程 A 试图在地址 0（我们称之为**虚拟地址**，virtual address）执行一次加载时，操作系统必须借助一些硬件支持，设法确保这次加载实际上不是去物理地址 0，而是去物理地址 320 KB（A 被加载到内存中的位置）。这正是内存虚拟化的关键，而世界上每一台现代计算机系统都建立在内存虚拟化之上。

13.4 目标

由此，我们来到了这一部分里操作系统的任务：虚拟化内存。不过，操作系统不只要虚拟化内存，还要做得漂亮。为了确保操作系统做到这一点，我们需要一些目标来指引方向。这些目标我们以前见过（想想引言那一章），以后还会再见到，但它们确实值得反复强调。

[2] “透明”的这种用法有时会让人困惑；有些学生以为“透明”意味着把一切都公开，就像政府应该做到的那样。而这里的意思恰恰相反：操作系统提供的假象不应该被应用程序察觉到。因此，按通常的用法，一个透明的系统指的是一个难以被察觉的系统，而不是一个按照《信息自由法》的规定回应查询请求的系统。

提示：隔离原则
隔离是构建可靠系统的一条关键原则。如果两个实体彼此适当隔离，就意味着其中一个出故障不会影响另一个。操作系统努力让各个进程彼此隔离，以此防止一个进程危害另一个。通过内存隔离，操作系统还能进一步确保运行中的程序无法影响底层操作系统的运行。有些现代操作系统把隔离做得更彻底，把操作系统的各个部分也彼此隔开。因此，这样的**微内核**（microkernel）[BH70, R+89, S+03] 可能比典型的宏内核设计更可靠。

虚拟内存（VM）系统的一个主要目标是**透明**（transparency）[2]。操作系统实现虚拟内存的方式应该让运行中的程序察觉不到。因此，程序不应该意识到内存被虚拟化了；相反，程序的表现就好像它拥有自己私有的物理内存一样。在幕后，操作系统（和硬件）完成所有工作，在许多不同的作业之间复用内存，从而实现这个假象。

虚拟内存的另一个目标是**效率**（efficiency）。操作系统应该努力让虚拟化尽可能**高效**，无论是在时间上（即不让程序运行得慢很多），还是在空间上（即不为支持虚拟化所需的数据结构占用太多内存）。在实现时间上高效的虚拟化时，操作系统将不得不依赖硬件支持，包括 TLB 这样的硬件特性（我们会在适当的时候学到它们）。

最后，虚拟内存的第三个目标是**保护**（protection）。操作系统应该确保**保护**各个进程不受彼此影响，也保护操作系统本身不受进程影响。当一个进程执行加载、存储或取指令时，它不应该能以任何方式访问或影响任何其他进程或操作系统本身的内存内容（也就是说，不能访问或影响其地址空间之外的任何东西）。因此，保护让我们能在进程之间实现**隔离**（isolation）这一特性；每个进程都应该运行在自己与世隔绝的茧里，不受其他出错的甚至恶意的进程的侵害。

在接下来的几章里，我们将重点探讨虚拟化内存所需的基本**机制**（mechanisms），包括硬件和操作系统的支持。我们还会研究操作系统中会遇到的一些比较重要的**策略**（policies），包括如何管理空闲空间，以及空间不足时把哪些页踢出内存。在这个过程中，我们会逐步帮你建立起对现代虚拟内存系统实际工作原理的理解[3]。

13.5 小结

我们已经介绍了操作系统的一个主要子系统：虚拟内存。虚拟内存系统负责为每个运行中的程序提供一个大的、稀疏的、私有的地址空间的假象；每个虚拟地址空间包含程序的全部指令和数据，程序可以通过虚拟地址引用它们。操作系统在硬件的大力帮助下，会把每一次这样的虚拟内存引用转换成物理地址，再交给物理内存，以取出或更新所需的信息。操作系统会同时为许多进程提供这项服务，确保程序之间互相保护，也保护操作系统本身。整个方法需要大量的机制（即许多底层的机器设施），还需要一些关键的策略才能运转；我们将自底向上，先描述关键的机制。那么，我们开始吧！

[3]或者，我们会说服你退掉这门课。不过先别急；只要你能撑过虚拟内存这一部分，你很可能就能一路撑到最后！

 © 2008–23, ARPACI-DUSSEAU

补充：你看到的每个地址都是虚拟的

写过打印指针的 C 程序吗？你看到的那个值（某个很大的数，通常以十六进制打印）就是一个**虚拟地址**。想知道你程序的代码放在哪里吗？你也可以把它打印出来，没错，只要能打印出来，它也是一个虚拟地址。事实上，作为用户级程序的程序员，你能看到的任何地址都是虚拟地址。只有操作系统通过它虚拟化内存的那些巧妙技术，才知道这些指令和数据值究竟位于机器物理内存的什么地方。所以千万别忘了：如果你在程序里打印一个地址，它是一个虚拟地址，是内存布局的一种假象；只有操作系统（和硬件）才知道真相。

下面这个小程序（`va.c`）打印出 `main()` 例程的位置（代码所在的地方）、`malloc()` 返回的堆分配值，以及栈上一个整数的位置：

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main(int argc, char *argv[]) {
4   printf("location of code : %p\n", main);
5   printf("location of heap : %p\n", malloc(100e6));
6   int x = 3;
7   printf("location of stack: %p\n", &x);
8   return x;
9 }
```

在 64 位的 Mac 上运行，得到如下输出：

```
location of code : 0x1095afe50
location of heap : 0x1096008c0
location of stack: 0x7fff691aea64
```

从中可以看出，代码位于地址空间的最前面，接着是堆，而栈则在这个巨大虚拟空间的另一端。所有这些地址都是虚拟的，操作系统和硬件会把它们转换成真实的物理位置，以便取出相应的值。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[BH70] "The Nucleus of a Multiprogramming System" by Per Brinch Hansen. Communications of the ACM, 13:4, April 1970. *The first paper to suggest that the OS, or kernel, should be a minimal and flexible substrate for building customized operating systems; this theme is revisited throughout OS research history.*

[CV65] "Introduction and Overview of the Multics System" by F. J. Corbato, V. A. Vyssotsky. Fall Joint Computer Conference, 1965. *A great early Multics paper. Here is the great quote about time sharing: "The impetus for time-sharing first arose from professional programmers because of their constant frustration in debugging programs at batch processing installations. Thus, the original goal was to time-share computers to allow simultaneous access by several persons while giving to each of them the illusion of having the whole machine at his disposal."*

[DV66] "Programming Semantics for Multiprogrammed Computations" by Jack B. Dennis, Earl C. Van Horn. Communications of the ACM, Volume 9, Number 3, March 1966. *An early paper (but not the first) on multiprogramming.*

[L60] "Man-Computer Symbiosis" by J. C. R. Licklider. IRE Transactions on Human Factors in Electronics, HFE-1:1, March 1960. *A funky paper about how computers and people are going to enter into a symbiotic age; clearly well ahead of its time but a fascinating read nonetheless.*

[M62] "Time-Sharing Computer Systems" by J. McCarthy. Management and the Computer of the Future, MIT Press, Cambridge, MA, 1962. *Probably McCarthy's earliest recorded paper on time sharing. In another paper [M83], he claims to have been thinking of the idea since 1957. McCarthy left the systems area and went on to become a giant in Artificial Intelligence at Stanford, including the creation of the LISP programming language. See McCarthy's home page for more info:* http://www-formal.stanford.edu/jmc/

[M+63] "A Time-Sharing Debugging System for a Small Computer" by J. McCarthy, S. Boilen, E. Fredkin, J. C. R. Licklider. AFIPS '63 (Spring), New York, NY, May 1963. *A great early example of a system that swapped program memory to the "drum" when the program wasn't running, and then back into "core" memory when it was about to be run.*

[M83] "Reminiscences on the History of Time Sharing" by John McCarthy. 1983. Available: http://www-formal.stanford.edu/jmc/history/timesharing/timesharing.html. *A terrific historical note on where the idea of time-sharing might have come from including some doubts towards those who cite Strachey's work [S59] as the pioneering work in this area.*

[NS07] "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation" by N. Nethercote, J. Seward. PLDI 2007, San Diego, California, June 2007. *Valgrind is a lifesaver of a program for those who use unsafe languages like C. Read this paper to learn about its very cool binary instrumentation techniques – it's really quite impressive.*

[R+89] "Mach: A System Software kernel" by R. Rashid, D. Julin, D. Orr, R. Sanzi, R. Baron, A. Forin, D. Golub, M. Jones. COMPCON '89, February 1989. *Although not the first project on microkernels per se, the Mach project at CMU was well-known and influential; it still lives today deep in the bowels of Mac OS X.*

[S59] "Time Sharing in Large Fast Computers" by C. Strachey. Proceedings of the International Conference on Information Processing, UNESCO, June 1959. *One of the earliest references on time sharing.*

[S+03] "Improving the Reliability of Commodity Operating Systems" by M. M. Swift, B. N. Bershad, H. M. Levy. SOSP '03. *The first paper to show how microkernel-like thinking can improve operating system reliability.*

 © 2008–23, ARPACI-DUSSEAU

作业（编程）

在这个作业中，我们只是学习几个有用的工具，用来查看基于 Linux 的系统上的虚拟内存使用情况。这只是简单提示一下能做些什么；要真正成为专家，你得自己深入钻研（一如既往！）。

问题

1. 你首先应该了解的 Linux 工具是非常简单的 `free`。先输入 `man free`，把它的整个手册页读一遍；它很短，别担心！
2. 现在运行 `free`，也许可以加上一些有用的参数（例如 `-m`，以 MB 为单位显示内存总量）。你的系统有多少内存？有多少是空闲的？这些数字和你的直觉相符吗？
3. 接下来，编写一个使用一定数量内存的小程序，叫作 `memory-user.c`。这个程序应该接受一个命令行参数：它要使用的内存大小（以 MB 为单位）。运行时，它应该分配一个数组，并不停地遍历这个数组，访问其中的每一项。程序应该无限期地这样做，或者也可以运行一段在命令行中指定的时间。
4. 现在，在运行 `memory-user` 程序的同时（在同一台机器的另一个终端窗口里）运行 `free` 工具。程序运行时，内存使用总量有什么变化？杀掉 `memory-user` 程序之后呢？这些数字和你的预期相符吗？用不同的内存使用量试一试。使用非常大的内存时会发生什么？
5. 我们再试一个工具，叫作 `pmap`。花点时间，仔细读一读 `pmap` 的手册页。
6. 要使用 `pmap`，你得知道你感兴趣的进程的**进程 ID**（process ID）。因此，先运行 `ps auxw`，查看所有进程的列表；然后挑一个有意思的，比如浏览器。这里也可以用你的 `memory-user` 程序（事实上，为了方便，你甚至可以让这个程序调用 `getpid()`，把自己的 PID 打印出来）。
7. 现在对其中一些进程运行 `pmap`，使用各种选项（比如 `-X`）来查看进程的许多细节。你看到了什么？与我们对代码/栈/堆的简单设想相比，一个现代的地址空间由多少个不同的部分组成？
8. 最后，对使用不同内存量的 `memory-user` 程序运行 `pmap`。你看到了什么？`pmap` 的输出和你的预期相符吗？

 © 2008–23, ARPACI-DUSSEAU